

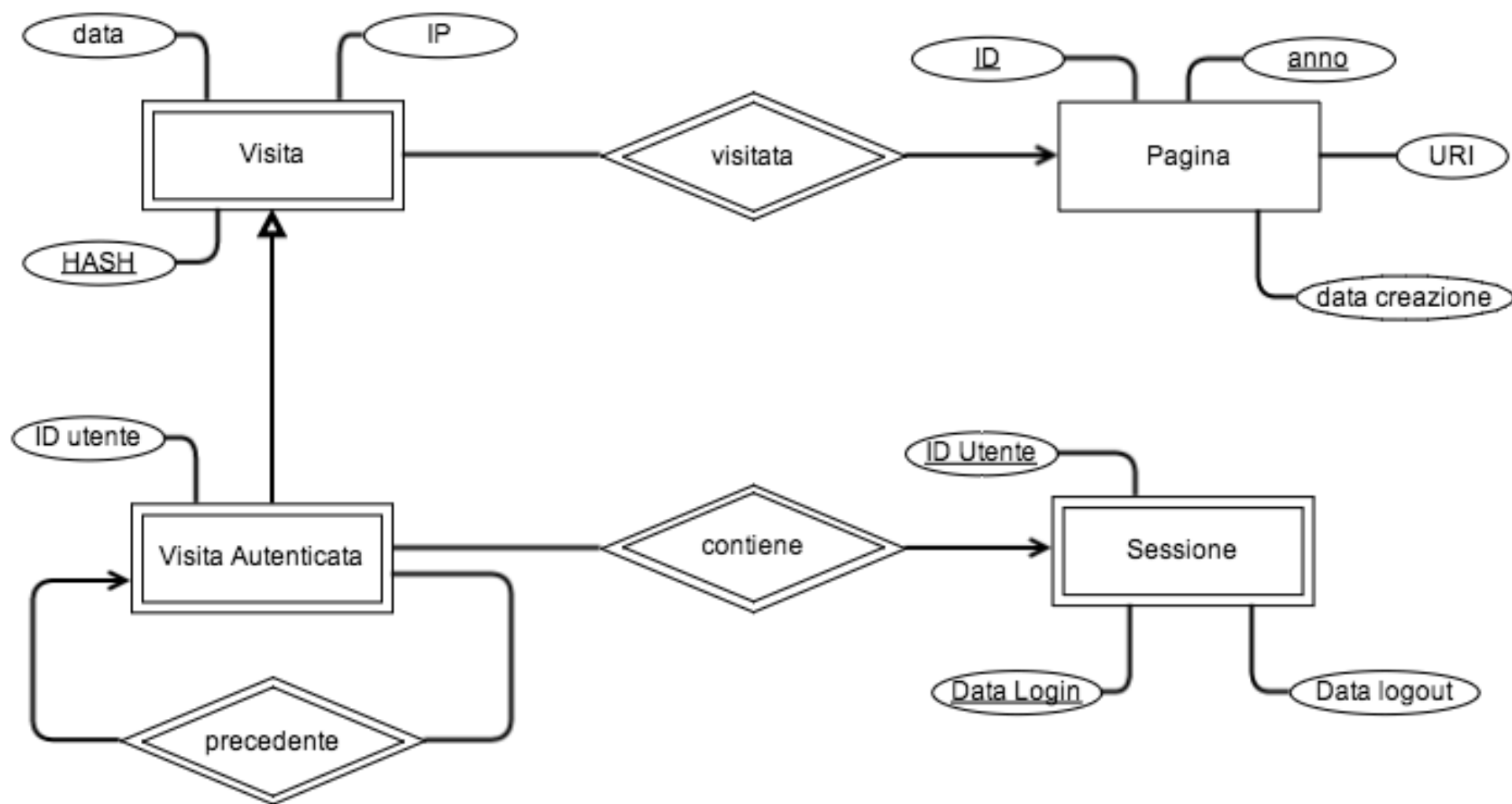
Suggerimenti per le soluzioni dell'Esame

11/02/2014

Corso di Basi di Dati
A.A. 2013/2014

Matteo Lissandrini
ml@disi.unitn.it

Esercizio 1 – Soluzioni (1/3)



Esercizio 1 – Soluzioni (2/3)

```
Pagina(id, anno, uri, data)
Visita (hash_sha256, ip, data, id_pag, anno_pag)
VisitaAutenticata (hash_visita, id_utente)
VisitaPrecedente (hash_visita, hash_visita_precedente)
Sessione (utente, data_login, data_logout)
VisiteSessione (utente_sessione, data_sessione, hash_visita)
```

```
CREATE TABLE exam.visita
(
    hash_sha256 CHAR(64) PRIMARY KEY,
    ip VARCHAR(15),
    data TIMESTAMP,
    id INT DEFAULT 0,
    anno INT DEFAULT 0,
    FOREIGN KEY(id, anno) REFERENCES exam.pagina (id, anno)
        ON UPDATE CASCADE ON DELETE SET DEFAULT
);
```

Esercizio 1 – Soluzioni (3/3)

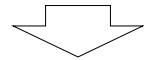
```
INSERT INTO exam.visita  
VALUES ( 'da6c ... 2f', '192.0.0.1', '12-02-2014 11:29:01', 12, 2013 );
```

Si deve assumere che la pagina esista

```
UPDATE exam.sessione  
SET data_logout = CURRENT_TIMESTAMP  
WHERE id_utente = 12 AND data_login = '12-02-2014 11:29:01';
```

Esercizio 2 – Soluzioni (1/3)

$\pi_{ur, mese, anno} ($
 $\sigma_{anno < 2005 \text{ OR } ((anno = 2005 \text{ AND } mese < 3) \text{ OR } (privata = TRUE \text{ AND } anno > 2010))} ($
 $\pi_{ur, mese, anno, privata}$ **(Pagina)**
)
)

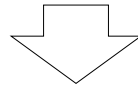


```
SELECT uri, mese, anno
FROM Pagina
WHERE anno < 2005
      OR (anno = 2005 AND mese < 3)
      OR (privata = TRUE AND anno_reg > 2010)
```

N.B. Per motivi tecnici qui non viene riportata la scrittura ad albero, ma è in genere caldamente consigliata

Esercizio 2 – Soluzioni (2/3)

$\tau_{c \text{ DESC}} \left(\gamma_{uri, COUNT(destinazione)} \rightarrow c \left(\pi_{uri, destinazione} \left(\right. \right. \right.$
 $\left. \left. \pi_{uri} \left(\sigma_{anno = 2010} \left(\pi_{uri, anno} (\text{Pagina}) \right) \right) \right. \right.$
 $\bowtie$
 $\left. \left. \rho_{link(uri, destinazione)} (\text{Link}) \right. \right.$
 $\left. \right)$
 $\left. \right)$



```
SELECT p.uri, COUNT(l.uri_destinazione) AS c_dest
  FROM pagina AS p
    JOIN link AS l
      ON p.uri = l.uri_sorgente AND p.anno = 2010
 GROUP BY p.uri
 ORDER BY c_dest DESC
```

Esercizio 2 – Soluzioni (3/3)

$\rho_v (\gamma_{\text{anno, uri, COUNT(utente)} \rightarrow n} \delta(\pi_{\text{anno, uri_pagina, utente}} (\text{Visita})))$
 $\neg_s$
 $\pi_{v1.anno, v1.uri, v1.n} ($
 $\rho_{v1} (\gamma_{\text{anno, uri, COUNT(utente)} \rightarrow n} \delta(\pi_{\text{anno, uri_pagina, utente}} (\text{Visita})))$
 $\bowtie$
 $v1.anno = v2.anno \text{ AND } v1.n < v2.n ($
 $\rho_{v2} (\gamma_{\text{anno, uri, COUNT(utente)} \rightarrow n} \delta(\pi_{\text{anno, uri_pagina, utente}} (\text{Visita})))$
 $)$

```
WITH v AS (  
    SELECT anno, uri, COUNT(DISTINCT utente) AS c  
    FROM Visita  
    GROUP BY anno, uri  
) SELECT v.anno, v.uri, v.c  
FROM v  
JOIN ( SELECT anno, MAX(c) AS m  
      FROM v  
      GROUP BY anno  
    ) AS maxs  
ON c.anno = maxs.anno AND v.c = maxs.m
```

Materiale aggiuntivo non necessario all'esercizio:

<https://periscope.io/blog/use-subqueries-to-count-distinct-50x-faster.html>

Esercizio 3 – Soluzioni (1/2)

```
DELETE FROM pagina AS p
WHERE anno < 2007
      AND p.uri NOT IN (
    (
      ( SELECT DISTINCT l.uri_destinazione AS uri
        FROM Link AS l )
      UNION
      ( SELECT DISTINCT l.uri_sorgente AS uri
        FROM Link AS l )
    )
)
```

Esercizio 3 – Soluzioni (2/2)

```
WITH RECURSIVE pg AS (  
    SELECT l.uri_destinazione AS uri  
    FROM Link AS l  
        JOIN (  
            SELECT p.uri  
            FROM Pagina AS p  
            WHERE autore = 1234  
        ) AS p  
    ON l.uri_sorgente = p.uri  
)  
UNION  
    (SELECT l.uri_destinazione AS uri  
    FROM pg  
        JOIN Link AS l  
        ON pg.uri = l.uri_sorgente  
    )  
)  
SELECT pg.uri  
FROM pg
```

Esercizio 4 – Soluzioni (1/1)

```
SELECT B, AVG(C) FROM R GROUP BY B
      NULL ; 6
      2    ; NULL
```

```
SELECT COUNT(*), COUNT(D) FROM Z
      3    ; 2
```

```
SELECT R.A, R.B, Z.A FROM R LEFT JOIN Z ON R.B=Z.D
      7    ; 2    ; 7
      7    ; 2    ; NULL
      NULL ; 2    ; 7
      NUL  ; 2    ; NULL
      4    ; NULL ; NULL
      4    ; NULL ; NULL
```

```
SELECT B, C FROM R WHERE A>3 AND C*0<1
      NULL ; 8
      NULL ; 4
```

Esercizio 5 – Soluzioni (1/2)

	{R}	{S}	{W}	{U}	R(a,b)	S(b,c)	W(c,d)	U(d,e)
Dimensione	200	300	100	200	R(10,50)	S(10,10)	W(5,20)	U(20,100)
Costo	0	0	0	0				
Migliore	{R}	{S}	{W}	{U}				
	{R,S}	{R,W}	{R,U}	{S,W}	{S,U}	{W,U}		
Dimensione	1200	20 000	40 000	3000	60 000	1000		
Costo	0	0	0	0	0	0		
Migliore	(R ⋈ S)	(R ⋈ W)	(R ⋈ U)	(S ⋈ W)	(S ⋈ U)	(W ⋈ U)		
	{R,S,W}	{R,S,U}	{R,W,U}	{S,W,U}				
Dimensione	12 000	240 000	200 000	30 000				
Costo	1200	1200	1000	1000				
Migliore	(R ⋈ S) ⋈ W	(R ⋈ S) ⋈ U	(R ⋈ (W ⋈ U))	S ⋈ (W ⋈ U)				

$((R \bowtie S) \bowtie W) \bowtie U) = 12\ 000 + 1200$
 $(R \bowtie S) \bowtie (W \bowtie U) = 1200 + 1000$
 $T(R \bowtie S) = T(R) * T(S) / V(R,b) = 200 * 300 / 50 = 1200$
 $T(W \bowtie U) = T(W) * T(U) / V(U,d) = 100 * 200 / 20 = 1000$

Esercizio 5 – Soluzioni (2/2)

$$a) \pi_b(\sigma_{U.d=10}(\pi_d(U)) \bowtie \pi_{b,d}(W \bowtie S))$$

$$T(\pi_d(U)) = 200$$

$$T(\sigma_{U.d=10}(\pi_d(U))) = 200/20 = 10$$

$$T(\pi_d(W \bowtie \pi_c(S))) = 3000$$

$$T(\sigma_{U.d=10}(\pi_d(U)) \bowtie \pi_d(W \bowtie \pi_c(S))) = 10 * 3000 / 20 = 1500$$

$$a) \pi_b(\sigma_{U.d=10}(\pi_d(U)) \bowtie W) \bowtie S$$

$$T(\sigma_{U.d=10}(\pi_d(U)) \bowtie W) = 10 * 100 / 20 = 50$$

$$T(\sigma_{U.d=10}(\pi_d(U)) \bowtie W \bowtie S) = 50 * 300 / 10 = 1500$$

Esercizio 6 – Soluzioni (1/2)

$R \cap_B S \quad B(R) < M$

Leggi il contenuto di R in $m_1 \dots m_M$
(costruendo una appropriata struttura di ricerca in memoria)

Leggi un blocco da S

Per ogni tupla s in S , se s si trova in uno dei blocchi m_i allora rimuovi la tupla da m_i e scrivi in output s .

Ripeti il precedente fino a che tutto S non è stato letto

Esercizio 6 – Soluzioni (2/2)

$R \cap_B S \quad B(R) > M$

Deve essere $B(R) + B(S) < M^2$

Si ordinano entrambe R e S rispetto ad ogni attributo

Si legge il primo blocco da R , R_b , e da S , S_b

Preso la prima tupla r da R_b si rimuovono tutte le tuple s da S_b tali per cui $r < s$
Se si raggiunge $r = s$ si rimuove r da R_b e s da S_b , si scrive in output r si ripete.

Se si raggiunge $r > s$ si rimuove r da R_b , si rimuove s da S_b e si ripete.

In qualsiasi momento se R_b o S_b risulta vuoto si legge il prossimo blocco.

Si continua per ogni blocco di R

Esercizio 7 – Soluzioni (1/2)

a)

T2 -> T3 : w2(B) -> w3(B)

T1 -> T3 : w1(C) -> r3(C) **nessun ciclo! -> Serializzabile**

Esecuzione seriale equivalente: **T1,T2,T3** oppure **T2,T1,T3**

b)

T1 -> T2 : r1(B) -> w2(B)

T3 -> T2 : r3(C) -> w2(C)

T2 -> T3 : w2(B) -> r3(B) **ciclo! -> NON Serializzabile**

Esercizio 7 – Soluzioni (2/2)

T1 $ul_1(B)$ $r_1(B)$ $xl_1(B)$ $w_1(B)$ $w_1(B)$ $ul_1(C)$ $r_1(C)$ $xl_1(C)$ $w_1(C)$ $r_1(B)$ $u_1(B)$ $u_1(C)$

T2 $sl_1(A)$ $r_2(A)$ $r_2(A)$ $xl_1(C)$ $w_2(C)$ $u_2(A)$ $u_2(C)$

T1	T2
	$sl(A); r(A);$
$ul(B); r(B)$	
	$r(A)$
$xl(B); w(B); w(B)$	
$ul(C); r(C); xl(C); w(C)$	
	$xl(C); \text{Denied}$
$r(B); u(B); u(C);$	
	$xl(C); w(C); u(A); u(C)$

Esercizio 8 – Soluzioni (1/1)

$A \rightarrow BC, CD \rightarrow AB, C \rightarrow D, AC \rightarrow F$

Deriva

$A \rightarrow B \quad A \rightarrow C \quad A \rightarrow D \quad A \rightarrow F$

Chiusure Transitive

$\{A\}^+ = \{A, B, C, D, F\}$

$\{C\}^+ = \{A, B, C, D, F\}$

$\{CD\}^+ = \{A, B, C, D, F\}$

$\{AC\}^+ = \{A, B, C, D, F\}$

$\{AE\}^+ = \{A, C, B, D, E, F\}$

$\{CE\}^+ = \{A, C, B, D, E, F\}$

Chiavi per R:

AE o CE

BCNF:

$R_1\{A, B, C\} \quad R_2\{A, D\} \quad R_3\{A, F\} \quad R_4\{A, E\}$